

Reviewer Pack — Mean Log-Bias of Internal Gates Lower-Bounds AC^0 PARITY Size

Entry 8f8f4d4c4a54 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 02:40:12 UTC

Mean Log-Bias of Internal Gates Lower-Bounds AC^0 PARITY Size

Entry ID: 8f8f4d4c4a54 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 02:40:12 UTC

1. Conjecture

Field A (mathematical branch): Information geometry — mean Kullback-Leibler divergence from Bernoulli(1/2) of the internal-gate bias empirical measure (Csiszár 1975 ‘I-divergence geometry of probability distributions’; Amari-Nagaoka 2000 ‘Methods of Information Geometry’; Cover-Thomas 2006). The functional uses log of empirical probabilities and per-gate $\min(p, 1-p)$, with NO polynomial F_q extension (log and min are not preserved under polynomial-ring lifts of the Boolean ring), so the bridge is Aaronson-Wigderson algebrization-safe. An arXiv search ‘mean gate-bias KL divergence’ AND (‘ AC^0 ’ OR ‘switching lemma’ OR ‘PARITY lower bound’) returns 0 direct hits and <5 adjacent papers (all on noise-sensitivity of the OUTPUT function in the Kahn-Kalai-Linial / Bourgain / O’Donnell tradition, never on the empirical measure of INTERNAL gate biases as a structural circuit invariant). Distinct from blacklisted Lasserre Moment-Rank Gap on accept set (truth-table side; NATURAL PROOFS-rejected), SoS Cone-Gram Stable Rank (Schatten- $1/\infty$ spectrum on a Gram matrix), Talagrand L^1 influence spread / Fourier spectral entropy / Mansour L^1 / Bonami $4/2$ / Bourgain NSM (all on the OUTPUT polynomial’s Fourier transform, not internal gate biases), and Dilworth width / Gromov hyperbolicity / Mostar / Forman-Ricci (all combinatorial-graph functionals on the DAG, never an information-geometric KL-from-uniform on Bernoulli pseudoexpectations). **Field B** (complexity object): AC^0 depth-d circuit size $s(C)$ for the n-bit PARITY function in the Håstad 1986 switching-lemma / Razborov-Smolensky / Beame 1994 regime, targeting the canonical $2^{\Theta(n)}\{1/(d-1)\}$ lower bound and the $V^0 \sqsubseteq V^0(\oplus)$ bounded-arithmetic separation. Gate biases p_g are exactly the SoS-degree-2 pseudoexpectations $\mathbb{E}[1_{\{g(x)=1\}}]$ under the uniform pseudo-distribution, so the invariant is a per-gate average of an SoS-2 functional — a Lasserre-pseudomoment-based circuit invariant in the Barak-Steurer 2014 / Grigoriev SoS hierarchy lineage.

Statement:

Let C be an AC^0 depth-d circuit on n inputs with m internal gates $g_1, \dots, g_m$ (including the output, excluding input literals). Let $p_g := \Pr_{\{x \sim U_n\}}[g(x)=1]$ (estimated by $N = \min(2^n, 2^{14})$ random samples) and define $\psi(C) := (1/m) \cdot \sum_g [-\log_2(2 \cdot \max(\min(p_g, 1-p_g), 1/(2m)))]$. CONJECTURE: (a) $\psi(C) \leq \log_2 m + 1$ for every AC^0 circuit C [universal cap]; (b) for every AC^0 depth-d circuit C exactly computing PARITY_n, $\psi(C) \geq 0.2 \cdot n^{\{1/(d-1)\}}$ [PARITY witness]. Combining (a)+(b) yields $\log_2 \text{size}(C) \geq 0.2 \cdot n^{\{1/(d-1)\}} - 1$.

Rationale (proposer’s reasoning):

Håstad’s switching lemma forces internal AC^0 gates whose downstream cone reaches PARITY’s perfectly balanced output to each carry exponentially small $\min(p_g, 1-p_g) \approx 2^{-n} \{1/(d-1)\}$; the mean-log-bias condenses this ‘bias bookkeeping’ into a single SoS-pseudoexpectation invariant. The invariant is RELATIONAL on the gate-bias multiset of C (two circuits with identical Boolean function can have very different gate-bias distributions because they wire intermediate gates differently), shielding from Razborov–Rudich natural-proofs and sidestepping the truth-table trap (j) that killed prior Lasserre Moment-Rank attempts. The natural S_n action on input bits permutes inputs but leaves the multiset $\{p_g\}$ fixed (uniform measure is S_n -invariant), giving the required non-trivial group equivariance.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ba0aa2ec2a6332ac

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds per (n,d) cell on ensembles $E1/E2/E3$, conjecture is SUPPORTED iff: (a) every circuit in all ensembles has $\psi(C) \leq \log_2(m)+1$; (b) $\text{Pearson}(\psi, n^{\{1/(d-1)\}}) \geq 0.9$ on $E1$ with OLS slope ≥ 0.15 ; (c) one-sided Mann-Whitney $E1 > E2$ and $E1 > E3$ each give $p < 0.01$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.86	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 10 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - gate bias KL divergence AC^0 circuit lower bound PARITY - internal gate empirical measure switching lemma pseudoexpectation Lasserre - average gate bias entropy bounded depth circuit PARITY Hastad

Top relevant hits considered: - [<http://arxiv.org/abs/2304.02770v2>] Tight Correlation Bounds for Circuits Between AC^0 and TC^0 - [<http://arxiv.org/abs/2604.11744v1>] KL Divergence Between Gaussians: A Step-by-Step Derivation for the Variational Autoencoder Objective - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/1205.6363v1>] What Should Developers Be Aware Of? An Empirical Study on the Directives of API Documentation - [<http://arxiv.org/abs/2103.17141v3>] Yoneda’s lemma for internal higher categories - [<http://arxiv.org/abs/q-bio/0703003v1>] Effect of Internal Viscosity on Brownian Dynamics of DNA Molecules in Shear Flow - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [<http://arxiv.org/abs/1712.05384v2>] Simulation of low-depth quantum circuits as complex undirected graphical models

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def evaluate_circuit(circuit, inputs):
        stack = []
        for gate in circuit:
            if isinstance(gate, tuple): # Input literal
                stack.append(gate[1])
            else: # Gate operation
                b = stack.pop()
                a = stack.pop()
                if gate == 'AND':
                    stack.append(a and b)
                elif gate == 'OR':
                    stack.append(a or b)
                elif gate == 'NOT':
                    stack.append(not b)
        return stack[0]

    def estimate_p_g(g, n):
        N = min(2**n, 16384)
        count = sum(evaluate_circuit(circuit, [random.randint(0, 1) for _ in range(n)]) for _ in range(N))
        p_g = Fraction(count, N)
        return p_g

    def psi(C):
        m = len(C)
        total = 0
        for g in C:
            p_g = estimate_p_g(g, n)
            min_p_g = min(p_g, 1 - p_g)
            max_min_p_g = max(min_p_g, Fraction(1, 2 * m))
            total += -math.log2(2 * max_min_p_g)
        return total / m

    def generate_hastad_circuit(n, d):
        if d == 2:
            # Full minterm DNF
            circuit = []
            for i in range(1 << n):
                term = True
                for j in range(n):
                    if (i >> j) & 1:
                        term &= ('NOT', j)
                else:
```

```

        term &= ('AND', j)
        circuit.append(term)
    elif d == 3:
        #  $\sqrt{n}$ -block tower
        block_size = int(math.sqrt(n))
        circuit = []
        for i in range(block_size):
            for j in range(block_size):
                if (i * block_size + j) < n:
                    circuit.append(('OR', ('AND', i), ('AND', j)))
    elif d == 4:
        # Recursive  $\sqrt[4]{n}$ -block
        block_size = int(math.sqrt(int(math.sqrt(n))))
        circuit = []
        for i in range(block_size):
            for j in range(block_size):
                for k in range(block_size):
                    if (i * block_size**2 + j * block_size + k) < n:
                        circuit.append(('OR', ('AND', i), ('AND', j), ('AND', k)))
    return circuit

def generate_random_circuit(n, m):
    circuit = []
    for _ in range(m):
        gate_type = random.choice(['AND', 'OR'])
        inputs = [random.randint(0, 1) for _ in range(random.randint(2, n))]
        if gate_type == 'NOT':
            inputs = [inputs[0]]
        circuit.append((gate_type,) + tuple(inputs))
    return circuit

def generate_or_circuit(n):
    return [('OR',)] * n

# Parameters
n_values = [6, 8, 10, 12, 16, 20, 24, 30]
d_values = [2, 3, 4]

results = []
for n in n_values:
    for d in d_values:
        # Håstad-style depth-d AC0 PARITY circuit
        circuit = generate_hastad_circuit(n, d)
        psi_value = psi(circuit)
        if psi_value > math.log2(len(circuit)) + 1:
            return {
                "metric_name": "psi",
                "metric_value": psi_value,
                "instances_tested": len(circuit),
                "conjecture_holds": False,
                "counterexample": f"Håstad-style depth-{d} circuit with n={n}"
            }
    results.append({"psi": psi_value, "n": n, "d": d})

# AC0 circuits of matching size computing random non-PARITY functions
circuit = generate_random_circuit(n, len(circuit))

```

```

psi_value = psi(circuit)
if psi_value > math.log2(len(circuit)) + 1:
    return {
        "metric_name": "psi",
        "metric_value": psi_value,
        "instances_tested": len(circuit),
        "conjecture_holds": False,
        "counterexample": f"Random AC0 circuit with n={n}"
    }
results.append({"psi": psi_value, "n": n, "d": d})

# Tiny AC0 computing the ORn function
circuit = generate_or_circuit(n)
psi_value = psi(circuit)
if psi_value > math.log2(len(circuit)) + 1:
    return {
        "metric_name": "psi",
        "metric_value": psi_value,
        "instances_tested": len(circuit),
        "conjecture_holds": False,
        "counterexample": f"ORn circuit with n={n}"
    }
results.append({"psi": psi_value, "n": n, "d": d})

# Compute statistics
mean_psi = sum(result["psi"] for result in results) / len(results)
std_psi = math.sqrt(sum((result["psi"] - mean_psi)**2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["psi"] <= math.log2(len(result["circuit"])))

return {
    "metric_name": "psi",
    "metric_value": mean_psi,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)

    mean_psi = sum(result["metric_value"] for result in results) / len(results)
    std_psi = math.sqrt(sum((result["metric_value"] - mean_psi)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_psi} std={std_psi} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next((seed for seed, result in enumerate(results) if not result["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")
    else:

```

```
print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_916b5b32.py", line 161, in <module>
    trial = run_trial(seed)
            ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_916b5b32.py", line 105, in run_trial
    circuit = generate_hastad_circuit(n, d)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_916b5b32.py", line 63, in generate_hastad_circuit
    term &= ('AND', j)
```

TypeError: unsupported operand type(s) for &=: 'bool' and 'tuple'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `generate_hastad_circuit` (`term &= ('AND', j)` misuses `&` on `bool` and `tuple`) before producing any data, so neither the universal cap nor the PARITY lower bound was empirically evaluated. | next: Fix the circuit construction bug (initialize `term` as a `tuple`/AST node rather than a `bool`, e.g., `term = ('AND',)` and append children) and re-run the 30-seed sweep over (n,d) cells for E1/E2/E3.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	335550
2	preregistration	claude_max	opus	0	0	7734
3	novelty	claude_max	opus	0	0	3171
4	novelty	claude_max	opus	0	0	8814
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21199
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11339
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26312
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19012
9	judge	claude_max	opus	0	0	4828

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 437959 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/8f8f4d4c4a54.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8f8f4d4c4a54.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/8f8f4d4c4a54.tar.gz (if generated)